Laboratorio Semana 9 – Escenario cliente-servidor ReservaFE

Oct 13, 2026 · @Alexis Flores Franco

Datos generales

Al terminar este laboratorio tu equipo tendrá definido el escenario cliente-servidor del proyecto del eje, con objetivos, alcance, actores, tipos de clientes, responsabilidades del servidor y protocolo. Además, habrás medido con un prototipo qué le pasa a un servidor cuando recibe muchas solicitudes simultáneas.

Campo	Detalle
Asignatura	Programación V (PROG 309, código 28186)
Eje temático	3. Patrón de Cliente/Servidor
Semana	9 (12 al 16 de octubre de 2026)
Tema	Requerimientos del escenario · Laboratorio: Definición del escenario cliente-servidor
Modalidad	Equipos de 3 o 4 integrantes (las Partes 4 y 5 se ejecutan en cada computadora)
Docente	Prof. Alexis Flores Franco

Organización de las sesiones

Sesión	Horario	Partes
Laboratorio 1	Jueves 15 de octubre, 7:00 a 8:35	Partes 1, 2 y 3 (análisis y definición)
Laboratorio 2	Viernes 16 de octubre, 7:00 a 8:35	Partes 4 y 5 (prototipo y mediciones)
Trabajo autónomo	Hasta la fecha de entrega	Retos y documento final

Objetivo

Definir el escenario de un sistema con arquitectura cliente-servidor, identificando sus actores, tipos de clientes, responsabilidades del servidor y protocolo de comunicación, y comprobar con un prototipo los efectos de la concurrencia y la saturación.

Competencias que desarrolla

- **Implementa patrones de diseño arquitecturales:** separas responsabilidades entre clientes y servidor en un caso real.

- **Aplica el software requerido para el desarrollo de patrones:** ejecutas y modificas un servidor y varios clientes en Java.
- **Capacidad para tomar decisiones:** eliges y justificas el protocolo y el reparto de responsabilidades.
- **Capacidad de investigación:** mides, comparas resultados y explicas su causa.

Requisitos previos

- ☐ **JDK 21** o superior (comprueba con `java -version`). El prototipo usa solo clases del JDK: no necesita Maven ni librerías, y cada archivo se ejecuta directamente con `java Archivo.java`.
- ☐ El IDE usado en los laboratorios anteriores (NetBeans, IntelliJ IDEA, Eclipse o VS Code con Extension Pack for Java) y Postman (o la extensión Thunder Client).
- ☐ El archivo `Prototipo_ReservaFE_Java.zip` publicado por el docente en la plataforma virtual.
- ☐ Una herramienta de diagramas (draw.io, Lucidchart o papel) para la Parte 2.

Parte 1 – Analizar el caso y redactar el escenario (30 min)

El caso: ReservaFE

Hoy los laboratorios de cómputo, los salones y el auditorio de la Facultad de Economía se reservan con una hoja impresa en secretaría. Esto produce tres problemas: choques de horario, nadie sabe qué está libre sin ir en persona y fuera del horario de oficina no se puede reservar.

La Facultad quiere una **API central de reservas** que sirva a varios clientes:

- una **página web** para estudiantes y docentes,
- una **app móvil** para estudiantes,
- un **panel administrativo** para la secretaría,
- una **pantalla en la entrada** del edificio que muestra las reservas del día.

El sistema también debe avisar por correo cuando se confirma o cancela una reserva. El auditorio requiere aprobación de la secretaría. Al inicio de cada semestre, cientos de docentes reservan laboratorios el mismo día.

Caso alternativo: tu equipo puede proponer otro sistema de reservas (canchas deportivas, tours en Boquete, citas de una clínica) si cumple tres condiciones: al menos **tres tipos de clientes**, un **recurso limitado** por el que compiten varios usuarios y un **pico de demanda** identificable. Debe aprobarlo el docente en esta sesión.

Paso 1.1 – Preguntas de análisis

Respóndanlas en equipo antes de redactar:

1. ¿Qué problema actual resuelve el sistema y para quién?
2. ¿Qué recurso es limitado y por qué varios usuarios compiten por él?
3. ¿Cuándo ocurre el pico de demanda y cuántas solicitudes estiman en ese momento?

4. ¿Qué queda **fuera** del sistema en esta primera versión (por ejemplo, pagos o integración con el sistema de matrícula)?

Paso 1.2 – Redactar la ficha del escenario

Completen la ficha en el documento del equipo:

Elemento	Contenido esperado
Nombre del sistema	Nombre corto y propósito en una frase
Problema actual	Situación y consecuencias, en 3 a 5 líneas
Objetivo general	Un objetivo que empiece con un verbo en infinitivo
Objetivos específicos	De 3 a 5, medibles
Alcance (incluye)	Funciones que sí se construirán en el eje
Fuera de alcance	Funciones que no se construirán y por qué
Recurso compartido	Qué se reserva y qué dato evita los choques (por ejemplo, espacio + fecha + hora)
Pico de demanda	Cuándo ocurre y estimación de usuarios y solicitudes

Punto de control 1: el docente revisa la ficha y aprueba el caso antes de pasar a la Parte 2.

Parte 2 – Actores, tipos de clientes y responsabilidades (35 min)

Un **actor** es quien interactúa con el sistema; un **tipo de cliente** es el programa que usa para hacerlo. Un mismo actor puede usar varios clientes, y un cliente puede ser otro sistema (por ejemplo, el servicio de correo).

Paso 2.1 – Tabla de actores y clientes

Completen la tabla para su caso. Incluyan al menos 4 actores y 3 tipos de clientes distintos. La primera fila es un ejemplo de ReservaFE.

Actor	Cliente que usa	Tipo (ligero o pesado)	Qué necesita del servidor	Frecuencia de uso
Estudiante	App móvil	Pesado: guarda sus reservas para verlas sin conexión	Ver disponibilidad, reservar salas de estudio, cancelar	Varias veces por semana

Actor	Cliente que usa	Tipo (ligero o pesado)	Qué necesita del servidor	Frecuencia de uso

Paso 2.2 – Reparto de responsabilidades

Para cada responsabilidad, decidan si va en el cliente, en el servidor o en ambos, y justifiquen. Recuerden la regla: **el cliente pide, el servidor decide**.

Responsabilidad	¿Cliente, servidor o ambos?	Justificación
Mostrar el calendario de disponibilidad		
Validar que la fecha no esté vacía		
Impedir dos reservas del mismo espacio, fecha y hora		
Verificar que el usuario puede reservar el auditorio		
Enviar el correo de confirmación		
Mostrar un mensaje cuando el servidor no responde		
Guardar el historial de reservas		
Agreguen 3 responsabilidades propias de su caso		

Paso 2.3 – Diagrama de contexto

Dibujen un diagrama con el **servidor** al centro, cada **tipo de cliente** alrededor y una flecha por cada comunicación, con una etiqueta de qué se pide (por ejemplo, “consultar disponibilidad”). Incluyan la base de datos y el servicio de correo como sistemas externos. Este diagrama se refina en la semana 11.

Punto de control 2: tablas completas y foto o exportación del diagrama.

Pregunta para el informe: ¿qué pasaría si la regla “no permitir choques de horario” solo existiera en la app móvil?

Parte 3 – Definir el protocolo de comunicación (25 min)

Paso 3.1 – Elegir y justificar el protocolo

Evalúen las opciones para su caso con esta matriz. Marquen de 1 (no cumple) a 3 (cumple bien).

Criterio	HTTP/REST	WebSocket	gRPC
Lo pueden usar todos nuestros tipos de clientes			
Se puede probar fácilmente (navegador, Postman)			
Necesitamos actualizaciones en tiempo real			
El equipo ya lo conoce			
Total			

Redacten la decisión en un párrafo: protocolo elegido, formato de datos (JSON), y si alguna parte necesitaría otro protocolo (por ejemplo, WebSocket para que la pantalla de la entrada se actualice al instante).

Paso 3.2 – Identificar los recursos

Un recurso es un sustantivo del negocio que el servidor administra. Para ReservaFE: `espacios`, `reservas`, `usuarios`. Listen los recursos de su caso con sus atributos principales:

Recurso	Atributos principales	Ejemplo en JSON
reservas	id, espacioId, fecha, hora, solicitante, estado	`{"id":1,"espacioId":3,"fecha":"2026-10-22","hora":"09:00","estado":"pendiente"}`

Paso 3.3 – Borrador de operaciones

Para cada actor de la Parte 2, escriban al menos 2 operaciones con su método, ruta, código de éxito y errores esperados. Sigan las convenciones REST: sustantivos en plural, versión o prefijo `/api`, acción en el método.

Actor	Operación	Método y ruta	Éxito	Errores esperados
Docente	Crear reserva	POST `/api/reservas`	201	400 datos inválidos · 409 horario ocupado

Actor	Operación	Método y ruta	Éxito	Errores esperados

Punto de control 3: matriz de decisión, párrafo de justificación y tabla de operaciones. Este borrador se convierte en el contrato de la API en la semana 11.

Parte 4 – Un servidor y tres tipos de clientes (35 min)

El prototipo es un servidor de reservas mínimo escrito en Java 21 con el `HttpServer` que trae el JDK. Lo usarás con tres clientes distintos para comprobar que todos hablan el mismo contrato HTTP/REST.

Paso 4.1 – Preparar el prototipo

1. Descarga `Prototipo_ReservaFE_Java.zip` de la plataforma virtual y descomprímelo.
2. Abre la carpeta `prototipo` en tu IDE. Tiene tres archivos:

Archivo	Rol
`Servidor.java`	Servidor HTTP con espacios, reservas y dos rutas de prueba de carga. Atiende cada solicitud en un hilo
`Cliente.java`	Cliente de consola con `HttpClient` que lista espacios y crea una reserva
`Carga.java`	Generador de solicitudes simultáneas para la Parte 5

3. Lee `Servidor.java` y ubica: dónde se valida la reserva (código 400), dónde se detecta el choque de horario (código 409), el bloque `synchronized`, y los métodos que atienden `/api/lento` y `/api/bloqueante`.

Esta es la parte del servidor que decide si una reserva se acepta. El bloque `synchronized` garantiza que solo un hilo a la vez verifique y guarde:

```
synchronized (reservas) {
    boolean ocupado = reservas.stream().anyMatch(r ->
            r.espacioId() == espacioId && r.fecha().equals(fecha) &&
r.hora().equals(hora));
    if (ocupado) {
        responder(ex, 409, "{\"error\":\"Ese espacio ya está reservado en esa fecha y
hora\"}");
        return;
    }
    nueva = new Reserva(reservas.size() + 1, espacioId, fecha, hora, solicitante);
    reservas.add(nueva);
}
responder(ex, 201, nueva.json());
```

Paso 4.2 – Iniciar el servidor

Abre una terminal en la carpeta `prototipo` (en el IDE o en la consola del sistema) y ejecuta:

```
java Servidor.java 4
```
El número `4` indica cuántos hilos atienden solicitudes. Debe mostrar `Servidor de reservas en http://localhost:3000 (hilos: 4, núcleos: N)`. **Anota cuántos núcleos tiene tu equipo**: lo necesitarás en la Parte 5. Deja esta terminal abierta: verás cada solicitud que llega y el nombre del hilo que la atendió.

Paso 4.3 – Cliente 1: navegador

Abre `http://localhost:3000/api/espacios` y luego `http://localhost:3000/api/estado`. Observa la terminal del servidor.

Paso 4.4 – Cliente 2: consola

Abre una **segunda** terminal en la misma carpeta y ejecuta dos veces, con nombres distintos:

```
java Cliente.java "Ana Perez"
java Cliente.java "Luis Gomez"
```
La primera ejecución debe responder **201** y la segunda **409**, porque ambas piden el Laboratorio 1 el 2026-10-20 a las 07:00.

Paso 4.5 – Cliente 3: Postman

Crea estas solicitudes y anota el código recibido:

#	Solicitud	Cuerpo JSON	Código esperado
1	`GET http://localhost:3000/api/reservas`	–	200
2	`POST http://localhost:3000/api/reservas`	`{"espacioId":2,"fecha":"2026-10-21","hora":"10:00","solicitante":"Tu nombre"}`	201
3	La misma solicitud 2 otra vez	Igual	409
4	`POST http://localhost:3000/api/reservas`	`{"espacioId":9,"fecha":"2026-10-21","hora":"10:00","solicitante":"X"}`	404
5	`POST http://localhost:3000/api/reservas`	`{"espacioId":2}`	400

Paso 4.6 – ¿Y si el servidor no está?

Detén el servidor con Ctrl + C y ejecuta `java Cliente.java "Prueba"`. El cliente debe mostrar `No se pudo conectar con el servidor en http://localhost:3000. ¿Está encendido?` en lugar de una excepción. Revisa en `Cliente.java` cómo se captura `ConnectException`. Vuelve a iniciar el servidor.

Punto de control 4: capturas de las respuestas 201 y 409 en la consola, de una solicitud en Postman y de la terminal del servidor con el registro de solicitudes e hilos.

Pregunta para el informe: los tres clientes son programas distintos. ¿Qué tienen en común que les permite usar el mismo servidor?

Parte 5 – Medir la saturación y la concurrencia (40 min)

Con `Carga.java` responderás el desafío de la clase con datos propios. El servidor tiene dos rutas que tardan 500 ms, pero de forma distinta:

- `/api/lento?ms=500` **espera** 500 ms (`Thread.sleep`), como una consulta a la base de datos: el hilo queda ocupado sin usar el procesador.
- `/api/bloqueante?ms=500` hace un **cálculo** de unos 500 ms que ocupa un núcleo del procesador.

Antes de cada experimento, **escribe tu predicción** del tiempo total usando la cantidad de hilos y de núcleos. Luego ejecútalo en la segunda terminal.

Ronda 1 – Servidor con 4 hilos (`java Servidor.java 4`)

#	Comando	Tu predicción (ms)	Tiempo total (ms)	Más lenta (ms)	Códigos
A	`java Carga.java "/api/lento?ms=500" 20`				
B	`java Carga.java "/api/bloqueante?ms=500" 20`				
E	`java Carga.java reservas 10`	–			

Experimento F – Un usuario inocente: ejecuta `java Carga.java "/api/lento?ms=500" 40` y, **mientras corre**, abre en el navegador `http://localhost:3000/api/espacios`. Anota cuánto tardó en cargar una consulta que normalmente es instantánea.

Ronda 2 – Servidor con hilos virtuales

Detén el servidor (Ctrl + C) e inícialo con `java Servidor.java virtual`. Cada solicitud recibe ahora su propio hilo virtual, un hilo liviano que Java 21 administra sin agotar la memoria.

#	Comando	Tu predicción (ms)	Tiempo total (ms)	Más lenta (ms)	Códigos
A2	`java Carga.java "/api/lento?ms=500" 20`				
D	`java Carga.java "/api/lento?ms=500" 200`				
B2	`java Carga.java "/api/bloqueante?ms=500" 20`				

Referencia (equipo del docente con 2 núcleos): A = 2,646 ms; B = 5,247 ms; E = un 201 y nueve 409; F = 4.8 s; A2 = 725 ms; D = 1,008 ms; B2 = 5,484 ms. Tus números cambiarán según los núcleos de tu equipo, pero la tendencia debe ser la misma.

Preguntas para el informe

1. En A, ¿por qué el tiempo total se acerca a 20 ÷ 4 × 500 ms? ¿Qué hacían las solicitudes 5 a 20 mientras tanto?
2. ¿Por qué los hilos virtuales mejoran tanto A2 y D, pero casi nada B2? Usa la palabra “núcleos” en tu respuesta.
3. ¿Qué te enseña el experimento F sobre el efecto de un solo endpoint lento en todos los usuarios?
4. En E, ¿qué papel cumple el bloque `synchronized`? ¿Qué podría fallar si hubiera dos copias del servidor, cada una con su propia lista en memoria?
5. Vuelve a la lluvia de ideas de la clase: ¿qué síntomas y soluciones se confirmaron con tus datos? Relaciónalos con al menos dos estrategias (escalar, hilos para esperar, caché, limitar la tasa, colas).

Punto de control 5: tablas completas con predicciones y resultados, capturas de A, B2 y E, y respuestas a las 5 preguntas.

Retos de programación (trabajo autónomo)

Los retos 1, 2 y 3 son obligatorios. El reto 4 otorga puntos extra y prepara el desafío de la semana 12 (clientes que manejan servidores que no responden).

Reto 1 – Cancelar una reserva

Agrega a `Servidor.java` la operación `DELETE /api/reservas/{id}`:

- Responde **204** si la reserva existía y se eliminó, y **404** si no existe.
- Pista: obtén el id con `ruta.matches("/api/reservas/\\d+")` y `Integer.parseInt(ruta.substring(ruta.lastIndexOf('/') + 1))`. Elimina con `reservas.removeIf(...)` dentro de `synchronized (reservas)`.
- Una respuesta 204 no lleva cuerpo: usa `ex.sendResponseHeaders(204, -1)` y luego `ex.close()`.

Pruébalo en Postman: crea una reserva, cancélala, cancélala otra vez (404) y vuelve a crearla (201, porque el horario quedó libre).

Reto 2 – Provocar y explicar una condición de carrera

Este reto muestra por qué existe el bloque `synchronized`. Trabaja en una **copia** de `Servidor.java`:

1. Cambia la declaración de la lista por `Collections.synchronizedList(new ArrayList<>())`.
2. En `crearReserva`, quita la palabra `synchronized (reservas)` y deja solo las llaves del bloque.
3. Justo antes de `reservas.add(nueva)`, agrega `try { Thread.sleep(50); } catch (InterruptedException e) { }` para simular una base de datos lenta.
4. Inicia con `java Servidor.java 4` y ejecuta `java Carga.java reservas 10`.

Resultado esperado: **varias respuestas 201** para el mismo espacio, fecha y hora (en el equipo del docente fueron 4). Revisa `GET /api/reservas`: el auditorio quedó reservado varias veces.

Preguntas: ¿por qué la lista sincronizada no fue suficiente? ¿Qué operaciones deben ocurrir juntas, sin que otro hilo se meta en medio?

Reto 3 – Proteger el servidor de la saturación

Imagina que la base de datos solo acepta **5 conexiones a la vez**. Modifica `Servidor.java` para que `/api/lento` atienda como máximo 5 solicitudes simultáneas. Las que excedan el límite deben responder de inmediato **503 Service Unavailable**, con el encabezado `Retry-After: 2`.

- Pista: crea `static final Semaphore conexionesBD = new Semaphore(5);`. Antes de esperar, usa `conexionesBD.tryAcquire()`; si devuelve `false`, responde 503. Libera el permiso con `release()` dentro de un bloque `finally`.
- Inicia con `java Servidor.java virtual` y repite A2 (20 solicitudes). Resultado esperado: 5 respuestas 200 y 15 respuestas 503 casi inmediatas.

Pregunta: ¿es mejor para el usuario recibir un 503 rápido o esperar varios segundos? ¿Qué debería hacer el cliente al recibir un 503?

Reto 4 – Un cliente resiliente (extra)

Modifica `Cliente.java` para que:

1. Cancele la solicitud si el servidor tarda más de 2 segundos: `HttpRequest.newBuilder(...).timeout(Duration.ofSeconds(2))` y captura `HttpTimeoutException`.
2. Reintente hasta 3 veces con espera creciente (1 s, 2 s y 4 s) cuando reciba 503 o un error de conexión.
3. Muestre mensajes claros al usuario en cada intento.

Pruébalo con el servidor apagado y con el reto 3 activo durante una carga.

Entrega de los retos: el código modificado en un repositorio de GitHub del equipo y capturas de las pruebas.

Entregables y evaluación

Se entregan en la plataforma virtual, en la fecha que indique el docente.

- ☐ **Documento de definición del escenario** (uno por equipo, PDF): ficha del escenario, tablas de actores y responsabilidades, diagrama de contexto, decisión de protocolo y borrador de operaciones (Partes 1 a 3). Es la base del documento de requerimientos de la semana 10.
- ☐ **Informe de laboratorio individual** (PDF) de las Partes 4 y 5, con la estructura de la plantilla institucional: portada, objetivo, marco teórico breve, procedimiento, capturas, resultados y conclusiones.
- ☐ **Enlace al repositorio de GitHub** con los retos.

Rúbrica (100 puntos + 10 extra)

Criterio	Puntos	Evidencia que se revisa
Ficha del escenario: problema, objetivos, alcance, recurso compartido y pico de demanda	15	Documento del equipo, Parte 1
Actores, tipos de clientes y reparto de responsabilidades justificado	15	Tablas y diagrama de contexto, Parte 2
Protocolo elegido con matriz y justificación; borrador de recursos y operaciones REST	15	Parte 3
Prototipo ejecutado con tres clientes y códigos verificados	10	Punto de control 4
Mediciones con predicciones y análisis de saturación y concurrencia	20	Tabla y respuestas de la Parte 5
Retos 1, 2 y 3 funcionando y	15	Repositorio, capturas y

Criterio	Puntos	Evidencia que se revisa
explicados		respuestas (5 puntos cada uno)
Calidad del informe: redacción, capturas y conclusiones	10	PDF individual
Extra: reto 4, cliente con tiempo límite y reintentos	+10	Repositorio y capturas

Criterio de cada punto: completo y correcto = 100 %; funciona con errores menores = 60 %; incompleto o no funciona = 0 %.

Solución de problemas

Síntoma	Causa probable	Solución
`java` no se reconoce como comando	El JDK no está instalado o no está en el PATH	Instala un JDK 21 (por ejemplo, Eclipse Temurin) y abre una terminal nueva
`error: class found on application class path` o no ejecuta el `.java`	Versión de Java antigua o hay archivos `.class` en la carpeta	Usa JDK 21 (`java -version`) y borra los `.class` de la carpeta
Error en `newVirtualThreadPerTaskExecutor`	JDK anterior a la versión 21	Actualiza a JDK 21 o usa solo el modo con número de hilos
`BindException: Address already in use`	Ya hay un servidor en el puerto 3000	Cierra la otra terminal, o inicia con otro puerto (`PORT=3001`; en PowerShell `$env:PORT=3001`) y usa `SERVIDOR=http://localhost:3001` en los clientes
El cliente dice “No se pudo conectar”	El servidor no está encendido	Inicia `java Servidor.java 4` en otra terminal
Las tildes salen como `?` o símbolos raros en la consola	Codificación de la consola de Windows	Ejecuta `chcp 65001` antes, o agrega `-Dstdout.encoding=UTF-8` después de `java`
Postman responde 400 aunque el JSON parece bien	Comillas simples, números entre comillas o campos faltantes	Usa Body → raw → JSON, comillas dobles y `espacioId` como número
La primera reserva ya da 409	Repetiste la prueba sin reiniciar el servidor	Los datos están en memoria: detén e inicia el servidor para

Síntoma	Causa probable	Solución
		empezar de cero
Los tiempos varían mucho entre ejecuciones	El equipo está ocupado o el JIT aún no se calentó	Cierra programas pesados y repite cada medición 3 veces; usa el promedio

Referencias

- Huaman, W. C. (2018). *Los 10 patrones comunes de arquitectura de software*. Medium.
- Conesa Caralt, J. (2010). *Introducción a .NET*. Editorial UOC.
- Java 21: HttpServer del JDK
- Java 21: HttpClient
- JEP 444: hilos virtuales
- MDN: códigos de estado de respuesta HTTP